Technical University of Denmark

DTU Compute
Department of Applied Mathematics and Computer Science

Introductory SQL, Part 2

Create Databases and Select Data

Chapter 3 in the Text Book

©**Anne Haxthausen** and Flemming Schmidt

These slides have been prepared by Anne Haxthausen, partly reusing/modifying slides by Flemming Schmidt, which again partly reused some slides by Silberschatz, Korth. Sudarshan, 2010.

Contents

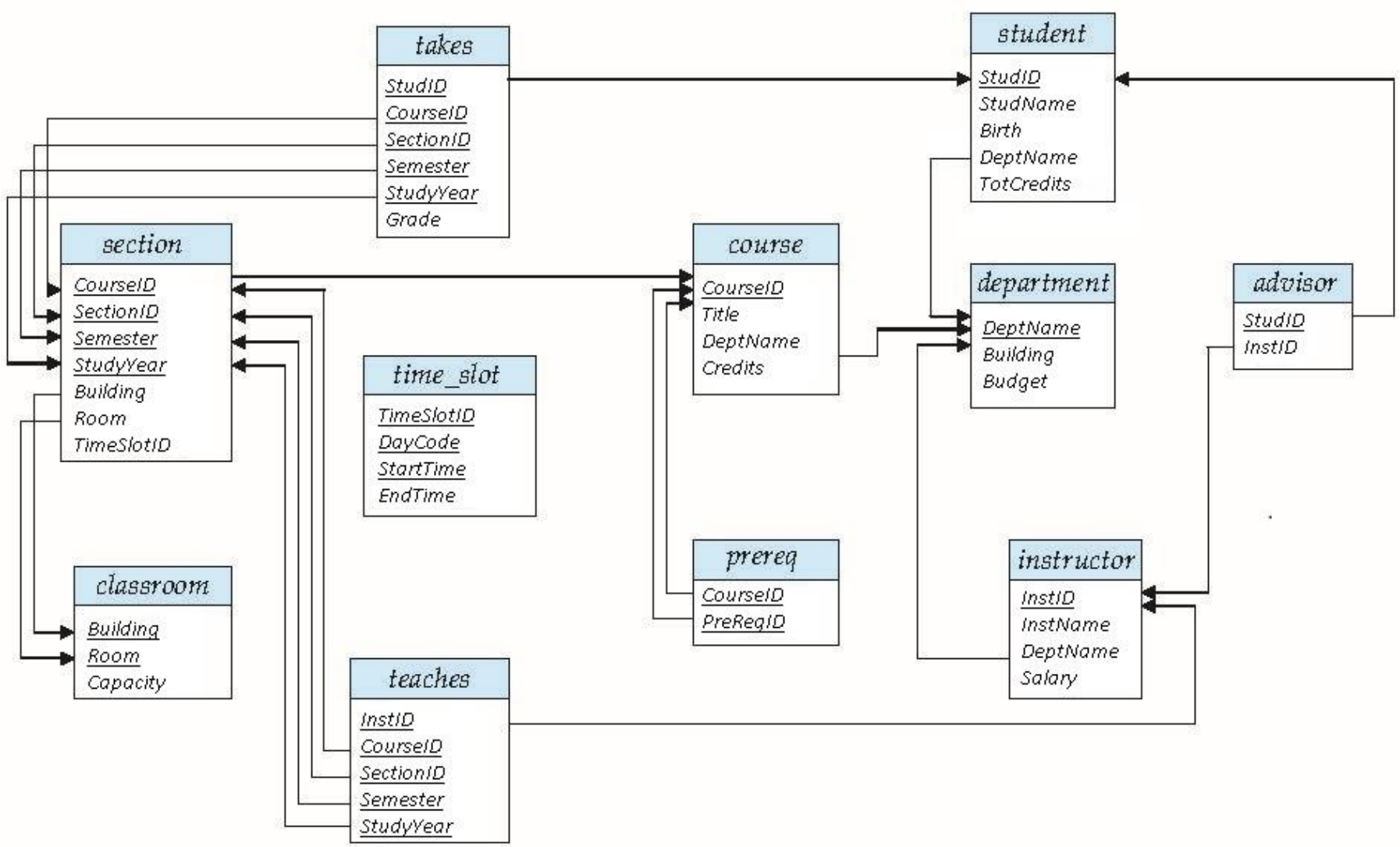
- University Database (running example)
- SQL Data, more details:
  - String Operations (book sec. 3.4.2)
  - NULL Values (book sec. 3.6)
- Advanced SQL Queries Using SELECT
  - Aggregate Functions + GROUP BY … HAVING … (book sec. 3.7)
  - Subqueries (book sec. 3.8, also used in 3.9)
  - Scalar subqueries (book sec. 3.8.7)
  - Conditions using IN, NOT IN (book sec. 3.8.1)
  - Conditions using op ALL, op SOME (book sec. 3.8.2)
  - Conditions using EXISTS, NOT EXISTS (book sec. 3.8.3)
- Composite SQL Queries Using UNION, INTERSECT and EXCEPT (book sec. 3.5)
- Demo Exercises & Exercises

University Database, revisited

- Database example used throughout the textbook and in this course!

1. Database Schema Diagram
2. Database Instance

Database Schema Diagram from the book, but with new names



Database Instance (1 of 4)

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Student

StudID	StudName	Birth	DeptName	TotCredits
00128	Zhang	1992-04-18	Comp. Sci.	102
12345	Shankar	1995-12-06	Comp. Sci.	32
19991	Brandt	1993-05-24	History	80
23121	Chavez	1992-04-18	Finance	110
44553	Peltier	1995-10-18	Physics	56
45678	Levy	1995-08-01	Physics	46
54321	Williams	1995-02-28	Comp. Sci.	54
55739	Sanchez	1995-06-04	Music	38
70557	Snow	1995-11-22	Physics	0
76543	Brown	1994-03-05	Comp. Sci.	58
76653	Aoi	1993-09-18	Elec. Eng.	60
98765	Bourikas	1992-09-23	Elec. Eng.	98
98988	Tanaka	1992-06-02	Biology	120

Advisor

StudID	InstID
12345	10101
44553	22222
45678	22222
00128	45565
76543	45565
23121	76543
98988	76766
76653	98345
98765	98345

Department

DeptName	Building	Budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Database Instance (2 of 4)

Teaches

InstID	CourseID	SectionID	Semester	StudyYear
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
10101	CS-101	1	Fall	2009
45565	CS-101	1	Spring	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
10101	CS-315	1	Spring	2010
45565	CS-319	1	Spring	2010
83821	CS-319	2	Spring	2010
10101	CS-347	1	Fall	2009
98345	EE-181	1	Spring	2009
12121	FIN-201	1	Spring	2010
32343	HIS-351	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Takes

StudID	CourseID	SectionID	Semester	StudyYear	Grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	NULL

Database Instance (3 of 4)

Course

CourseID	Title	DeptName	Credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

PreReq

CourseID	PreReqID
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Database Instance (4 of 4)

Section

CourseID	SectionID	Semester	StudyYear	Building	Room	TimeSlotID
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

TimeSlot

TimeSlotID	DayCode	StartTime	EndTime
A	M	08:00:00	08:50:00
A	W	08:00:00	08:50:00
A	F	08:00:00	08:50:00
B	M	09:00:00	09:50:00
B	W	09:00:00	09:50:00
B	F	09:00:00	09:50:00
C	M	11:00:00	11:50:00
C	W	11:00:00	11:50:00
C	F	11:00:00	11:50:00
D	M	13:00:00	13:50:00
D	W	13:00:00	13:50:00
D	F	13:00:00	13:50:00
E	T	10:30:00	11:45:00
E	R	10:30:00	11:45:00
F	T	14:30:00	15:45:00
F	R	14:30:00	15:45:00
G	M	16:00:00	16:50:00
G	W	16:00:00	16:50:00
G	F	16:00:00	16:50:00
H	W	10:00:00	12:30:00

Classroom

Building	Room	Capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Note: When writing and testing SQL queries it is useful to have a print of the Diagram and the Database Instance.

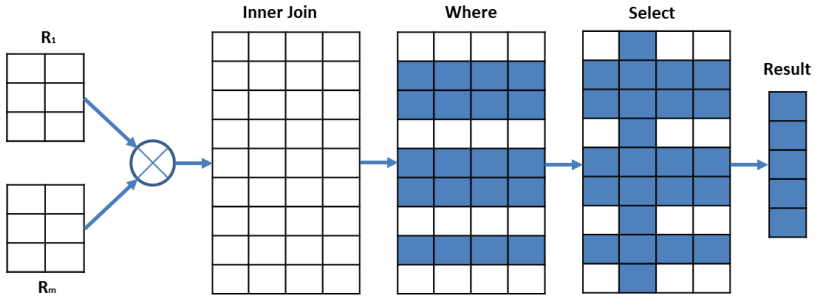
SQL Data Queries

- Basic Query Structure
  - SQL Data Query Language provides the ability to query data
  - The result of a SQL Query is a table

- A typical *basic* SQL query has the form:

SELECT $A_1, A_2, ..., A_n$
FROM $r_1, r_2, ..., r_m$
WHERE P;

  - A_i represents an attribute
  - r_i represents a relation (can e.g. be an id or a [natural] join expression)
  - P is a (row) predicate over attribute names. For each row it is either true or false.



String Operations (section 3.4.2)

- Strings are sequences of characters enclosed with quotes
  - E.g. 'Anne'
  - How to include ' and \ in strings: 'Anne\'s' and 'A\\B'
- SQL supports a variety of string operations such as
  - =, <>, <, >: are case sensitive according to the SQL standard, but not in MySQL and MariaDB, where e.g. 'anne' = 'Anne' evaluates to 1 (true)
  - Concatenation, using **CONCAT**
  - Finding string length, extracting substrings, etc, see the DBMS manual.
  - Pattern matching, using **LIKE**

- Examples

```
SET @TeachingHistory = CONCAT('CS-101', ', ' , 'CS-315' , ', ' , 'CS-347'); # @ defines a variable
SELECT @TeachingHistory;
```

@TeachingHistory
CS-101, CS-315, CS-347

String Operations: Pattern Matching

- **LIKE** is a string-matching operator for comparisons of character strings:
  - Syntax: string-expr **LIKE** string-pattern
  - Can be used where a Boolean expression is expected, e.g. in a WHERE clause.
  - Returns 1 (true) if string-expr matches string-pattern, otherwise 0 (false).
  - The string-pattern can use two special characters:
    - The % character matches any substring (of 0 - n characters).
    - The _ character matches any (single) character.
- Pattern matching examples
  - 'Anne' matches 'Anne' , but not 'Hanne'
  - 'Intro%' matches any string beginning with “Intro”, e.g. 'Introduction'
  - '%duc%' matches any string containing “duc” as a substring”, e.g. 'Introduction'
  - '_ _ _' matches any string of exactly three characters, e.g. 'Ann'
  - '_ _ _ %' matches any string of at least three characters, e.g. 'Hanne'

String Operations: Pattern Matching

- Patterns are case sensitive according to the SQL standard, but not in MySQL and MariaDB where e.g.
  - 'anne' **LIKE** 'Anne'

  will evaluate to 1.
- Query Example using pattern matching: Find the names of all instructors whose name includes the substring “ri”.

```
SELECT InstName FROM Instructor
WHERE InstName LIKE '%ri%';
```

InstName
Califieri
Crick
Srinivasan

NULL Values in Expressions (section 3.6)

- A row can have a **NULL** value for some attributes.
  - **NULL** signifies an unknown value or that a value does not exist.

- Arithmetic Expressions containing **NULL** return **NULL.**
  - Example: 5 + **NULL** returns **NULL**

- The predicate **IS NULL** can be used to check for Null.
  - Example: Find all instructors whose salary is unknown.

    SELECT InstName **FROM** Instructor **WHERE** Salary **IS NULL**;

- The predicate **IS NOT NULL** can be used to check for not Null

NULL and Three Valued Logic

- Comparison Expressions containing **NULL** return **UNKNOWN**
  - Examples:
    5 < **NULL**, **NULL** <> **NULL**, and **NULL** = **NULL** all evaluate to **UNKNOWN**
- Three-valued (conditional) logic using the truth value **UNKNOWN**

OR	FALSE	TRUE	UNKNOWN
FALSE	FALSE	TRUE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
UNKNOWN	UNKNOWN	TRUE	UNKNOWN

AND	FALSE	TRUE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
TRUE	FALSE	TRUE	UNKNOWN
UNKNOWN	FALSE	UNKNOWN	UNKNOWN

NOT	FALSE	TRUE	UNKNOWN
Result	TRUE	FALSE	UNKNOWN

- A **WHERE** or **HAVING** clause predicate is treated as **FALSE**, if it evaluates to **UNKNOWN**
- The predicate **IS [NOT] UNKNOWN:**
  - "P **IS UNKNOWN**" evaluates to TRUE, if predicate P evaluates to **UNKNOWN**

Treatment of NULL By SELECT DISTINCT

- **SELECT DISTINCT … FROM …** removes duplicate rows.
  - When comparing two tuples, it treats **NULL** as being equal to **NULL** (although **NULL = NULL** returns **UNKNOWN**).
  - **Example 1:** If there are two rows (1, **NULL**) and (1, **NULL**), one of them are removed.
  - **Example 2:** If there are two rows (1, 2) and (1, **NULL**), both are kept.

Aggregate Functions (section 3.7)

- These functions operate on the set of values in a column of a given attribute A and return a value:

  AVG(A): Average of values in the A-column
  MIN(A): Minimum of values in the A-column
  MAX(A): Maximum of values in the A-column
  SUM(A): Sum of values in the A-column
  COUNT(A): Number of values in the A-column

- They can be used in SELECT clauses and HAVING clauses.
- Example:

  SELECT AVG(Salary), **MIN**(Salary), **Max**(Salary), **SUM**(Salary), **COUNT**(Salary)
  FROM Instructor;

AVG(Salary)	MIN(Salary)	Max(Salary)	SUM(Salary)	COUNT(Salary)
75416.666667	40000.00	95000.00	905000.00	12

- **COUNT**(*): counts the number of rows in a relation/table.
- Example: **SELECT COUNT**(*) **FROM** Course;

COUNT(*)
13

NULL Values and Aggregates

- Find the total of all instructor salaries

  SELECT SUM(Salary) **FROM** Instructor;

  - This statement ignores **NULL** amounts and sums the rest.
  - Result is **NULL**, only if ALL salaries are **NULL.**

- **AVG**(A), **MIN**(A), **MAX**(A), **SUM**(A), and **COUNT**(A) ignore rows where A is NULL.
- What if an A column only has NULL values or is empty?
  - **COUNT(**A**)** returns 0,
  - **MIN(**A**)**, **MAX(**A**)**, **SUM(**A**)**, and **AVG(**A**)** return **NULL**
- **COUNT**(*) does not ignore rows with NULL values.

Duplicates and Aggregates

- Aggregate functions take duplicate values of the aggregate attribute into account.
- Example: Find the average salary of instructors in the Computer Science department

```
SELECT AVG(Salary) FROM Instructor
WHERE DeptName='Comp. Sci.';
```

- To ignore duplicate values, use the **DISTINCT** keyword.
- Example: Find the total number of instructors who teach a course in the Spring 2010 semester

```
SELECT COUNT(DISTINCT InstID) FROM Teaches
WHERE Semester='Spring' AND StudyYear=2010;
```

GROUP BY

- Find the name and average instructor salary for each department:

SELECT * **FROM** Instructor **ORDER BY** DeptName;

InstID	InstName	DeptName	Salary
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
83821	Brandt	Comp. Sci.	92000.00
45565	Katz	Comp. Sci.	75000.00
98345	Kim	Elec. Eng.	80000.00
76543	Singh	Finance	80000.00
12121	Wu	Finance	90000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
15151	Mozart	Music	40000.00
33456	Gold	Physics	87000.00
22222	Einstein	Physics	95000.00

SELECT DeptName, **AVG**(Salary)
FROM Instructor
GROUP BY DeptName**;**

DeptName	AVG(Salary)
Biology	72000.000000
Comp. Sci.	77333.333333
Elec. Eng.	80000.000000
Finance	85000.000000
History	61000.000000
Music	40000.000000
Physics	91000.000000

Aggregation with GROUP BY

- Study the table Testscores and possible Aggregates

SELECT * **FROM** Testscores;

Student	Test	Score
Brandt	A	47
Brandt	B	50
Brandt	C	NULL
Brandt	D	NULL
Chavez	A	52
Chavez	B	45
Chavez	C	53
Chavez	D	NULL

```
SELECT Student,
   COUNT(Score) AS n,
   SUM(Score) AS Total,
   AVG(Score) AS Average,
   MIN(Score) AS Lowest,
   MAX(Score) AS Highest
   FROM Testscores GROUP BY Student;
```

Student	n	Total	Average	Lowest	Highest
Brandt	2	97	48.5000	47	50
Chavez	3	150	50.0000	45	53

SELECT COUNT(*) **FROM** Testscores;

COUNT(*)
8

HAVING

- Find the name and average salary for those departments whose average salary is greater than 65000.

SELECT DeptName, **AVG**(Salary) **FROM** Instructor
GROUP BY DeptName **HAVING AVG**(Salary) > 65000;

DeptName	AVG(Salary)
Biology	72000.000000
Comp. Sci.	77333.333333
Elec. Eng.	80000.000000
Finance	85000.000000
Physics	91000.000000

- Note: Predicates in the **HAVING** clause are applied after the formation of groups, whereas predicates in the **WHERE** clause are applied before forming groups.
  So, **WHERE** is working on each Row and **HAVING** on each Group result.

GROUP BY

- Typical form

```
SELECT attribute-list1, aggregations
FROM … WHERE P1
GROUP BY attribute-list2 HAVING P2
```

  - Attributes in attribute-list1 must be present in attribute-list2.
  - Usually attribute-list1 = attribute-list2.
  - In P2 there can be aggregations (like in the SELECT clause). Attributes that appear in P2 without being aggregated over, must be present in attribute-list2.

More General SQL Queries

- A more general form of SELECT queries:

SELECT attributes
FROM r_1, r_2, ..., r_m [**WHERE** P1]
[**GROUP BY** group-spec [**HAVING** P2]]
[**ORDER BY** order-spec];

- Meaning:
  1. Calculate the relation r represented by r_1, r_2, ..., r_m and remove rows from r not satisfying P1.
  2. Arrange the selected rows into groups having the same values for group-spec and remove groups not satisfying P2.
  3. For each group calculate the attributes: this gives one tuple/row for each group.
  4. Order the rows according to the order-spec.

Subqueries (section 3.8)

- What:
  - A SELECT statement is said to be a subquery, if it occurs nested inside another statement.
- Where:
  - A SELECT statement can be used to represent a relation in the WHERE, FROM and HAVING clauses of another (outer) SELECT statement.
- Examples:
  - Will be shown on the following slides.
- Purpose:
  - They provide alternative ways to perform operations that would otherwise require complex joins and unions.

Scalar Subqueries (section 3.8.7)

- If a subquery returns a 1x1 relation then it is said to be a scalar subquery.
- A scalar subquery can be used where a single value is expected (in the SELECT, WHERE, FROM and HAVING clauses). Example:
  - **SELECT** InstName **FROM** Instructor

    WHERE Salary > (**SELECT AVG**(Salary) **FROM** Instructor);
  - For an instance, this corresponds to:

    … **FROM**

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

WHERE Salary >

AVG(Salary)
'74833.333333'

  - Result table: ---------->

InstName
Wu
Einstein
Gold
Katz
Singh
Brandt
Kim

Set Membership Conditions: IN and NOT IN (section 3.8.1)

- IN and NOT IN can be used in a WHERE/HAVING clause to form a condition.
- Typical forms:
  - **SELECT**…, A_i,… **FROM** …
    WHERE A_i [**NOT**] **IN** (*value1*, *value2*, ...);
  - **SELECT**…, A_i,… **FROM** …
    WHERE A_i [**NOT**] **IN** (**SELECT** B_j **FROM** … **WHERE** …); #often $A_i = B_j$
- Example: select the names of instructors whose name is neither Mozart, Einstein:
  - **SELECT DISTINCT** InstName **FROM** Instructor
    WHERE InstName **NOT IN** ('Mozart', 'Einstein') ;
- Example: Find courses offered both in Fall 2009 and in Spring 2010

```
SELECT DISTINCT CourseID FROM Section
WHERE
   Semester='Fall' AND StudyYear=2009 AND
   CourseID IN
            (SELECT CourseID FROM Section WHERE Semester='Spring' AND StudyYear=2010);
```

Conditions Using ALL and SOME (section 3.8.2)

- Can be used in a WHERE/HAVING clause to form a condition.
- Typical forms:
  - *A op* **ALL** (**SELECT** *A* **FROM** … **WHERE** …);
  - *A op* **SOME** (**SELECT** *A* **FROM** … **WHERE** …);

  where *op* can be: $<$, $\leq$, $>$, $=$, $<>$
- It checks whether *A op v* is true for all/some of the values *v* in the column specified by the SELECT.

SOME

- Find names of instructors with salary greater than that of some (at least one) instructor in the Finance department.

```
SELECT InstName FROM Instructor
WHERE Salary > SOME (SELECT Salary FROM Instructor
                     WHERE DeptName = 'Finance');
```

- An instance of this corresponds to:

SELECT InstName **FROM**

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

WHERE Salary > **SOME**

Salary
90000
80000

- Which gives:

InstName
Wu
Einstein
Gold
Brandt

SOME conditions

- *A op* **SOME** *r* $\Leftrightarrow \exists\, v \in r$: (A op *v*)
  Where *r* is a relation with one attribute and
  op can be: $<,\ \leq,\ >,\ =,\ \neq$

(5 < **SOME** | 0 / 5 / 6 |) = true

(5 < **SOME** | 0 / 5 |) = false

(5 = **SOME** | 0 / 5 |) = true

(5 ≠ **SOME** | 0 / 5 |) = true (since 0 ≠ 5)

(= **SOME**) ≡ **IN**

However, (≠ **SOME**) ≢ **NOT IN**

ALL conditions

- *A op* **ALL** *r* $\Leftrightarrow$ $\forall$ *v* $\in$ *r :* (A op *v)*

(5 < **ALL** | 0 / 5 / 6 |) = false

(5 < **ALL** | 6 / 10 |) = true

(5 = **ALL** | 4 / 5 |) = false

(5 $\neq$ **ALL** | 4 / 6 |) = true

($\neq$ **ALL**) $\equiv$ **NOT IN**

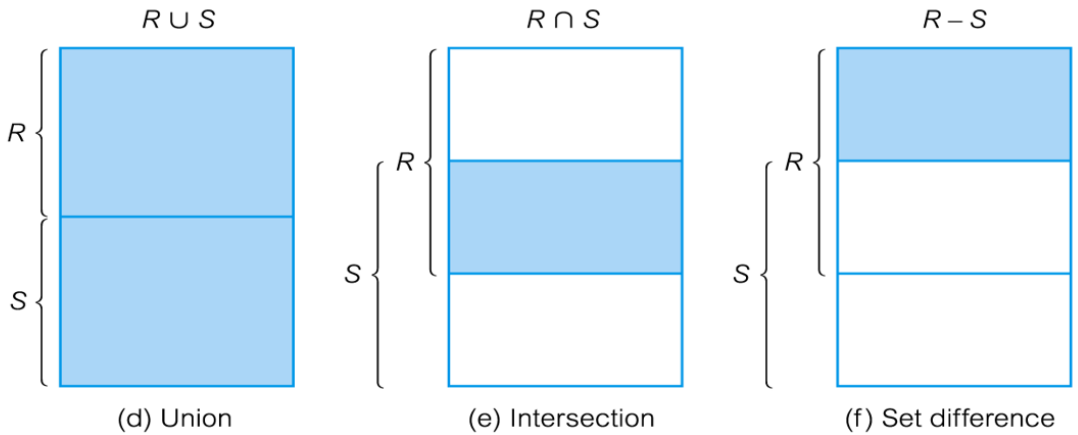
But: (= **ALL**) $\not\equiv$ **IN**

Conditions Using EXISTS and NOT EXISTS (section 3.8.3)

- Can be used in a WHERE/HAVING clause to form a condition:
  - [**NOT**] **EXISTS** (**SELECT** … **FROM** … **WHERE** …);
- It checks whether the relation is [not] non-empty
- Example: Find all courses taught in both the Fall 2009 semester and in the Spring 2010 semester.
  - **SELECT** CourseID **FROM** Section **AS** S
    WHERE Semester='Fall' **AND** StudyYear=2009 **AND**
    EXISTS (**SELECT** *
    FROM Section AS T
    WHERE Semester = 'Spring' **AND** StudyYear= 2010
    AND S.CourseId = T.CourseId);

Compound Queries Using UNION, INTERSECT & EXCEPT (section 3.5)

- Set Operations **UNION**, **INTERSECT** & **EXCEPT** have
  - **Arguments:** two SELECT statements (denoting two relations R and S).
    - R and S must be compatible: Number and types of the attributes of R and S must be the same.
  - **Result:** is the relation consisting of the union/intersection and set difference of the tuples in R and S.
    - **UNION** and **INTERSECT** removes duplicates in the result.
    - **EXCEPT** removes duplicates in its arguments before the operation is done.



(d) Union (e) Intersection (f) Set difference

- Applications of set operations constitute a query (like SELECT statements do).
- They can't appear inside a SELECT statement.

UNION, INTERSECT and EXCEPT

- Find courses offered in Fall 2009 or in Spring 2010

```
(SELECT CourseID FROM Section WHERE Semester='Fall' AND StudyYear=2009)
 UNION
(SELECT CourseID FROM Section WHERE Semester='Spring' AND StudyYear = 2010);
```

R

CS-101
CS-347
PHY-101

S

CS-101
FIN-201
MU-199
HIS-351
CS-319
CS-319
CS-315

R UNION S

CS-101
CS-347
PHY-101
FIN-201
MU-199
HIS-351
CS-319
CS-315

R INTERSECT S

CS-101

R EXCEPT S

CS-347
PHY-101

INTERSECT and EXCEPT

- Find courses offered both in Fall 2009 and in Spring 2010

```
(SELECT CourseID FROM Section WHERE Semester='Fall' AND StudyYear=2009)
 INTERSECT
(SELECT CourseID FROM Section WHERE Semester='Spring' AND StudyYear = 2010);
```

can be expressed by:

```
SELECT DISTINCT CourseID FROM Section
WHERE Semester='Fall' AND StudyYear=2009 AND CourseID
   IN (SELECT CourseID FROM Section WHERE Semester='Spring' AND StudyYear=2010);
```

- Find courses offered in Fall 2009 but not in Spring 2010

```
(SELECT CourseID FROM Section WHERE Semester='Fall' AND StudyYear=2009)
EXCEPT
(SELECT CourseID FROM Section WHERE Semester='Spring' AND StudyYear = 2010);
```

can be expressed by:

```
SELECT DISTINCT CourseID FROM Section
WHERE Semester='Fall' AND StudyYear=2009 AND CourseID
  NOT IN (SELECT CourseID FROM Section WHERE Semester='Spring' AND StudyYear=2010);
```

Summary

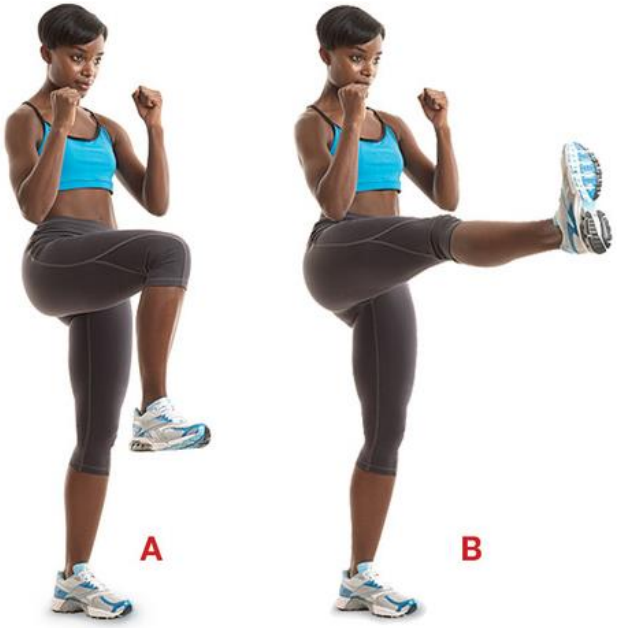
- Data Queries:
  - SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY …
  - … UNION …,
  - … INTERSECT …,
  - … EXCEPT …

Readings

- In Database Systems Concepts (6th and 7th edition): 3.4.2, 3.5, 3.6, 3.7, 3.8 (except 3.8.4, 3.8.6, 3.8.8), and those parts of 3.9, not covered last time. In 7th edition: skip also Note 3.2 on page 97 and Note 3.3 on page 108.

- Pay special attention to the Summary

Demo Exercises



Demo Exercises clarify ideas and concepts from the Lecture to provide you with good Database Skills.

Do the Demo Exercises. Solutions can be found on a later slide.

Advanced SQL Queries and Data Manipulation

The following SQL queries should be run against the University database.

3.1.1 SELECT with COUNT aggregation and GROUP BY

Find the number of enrolments of each course section that was offered in Fall 2009.

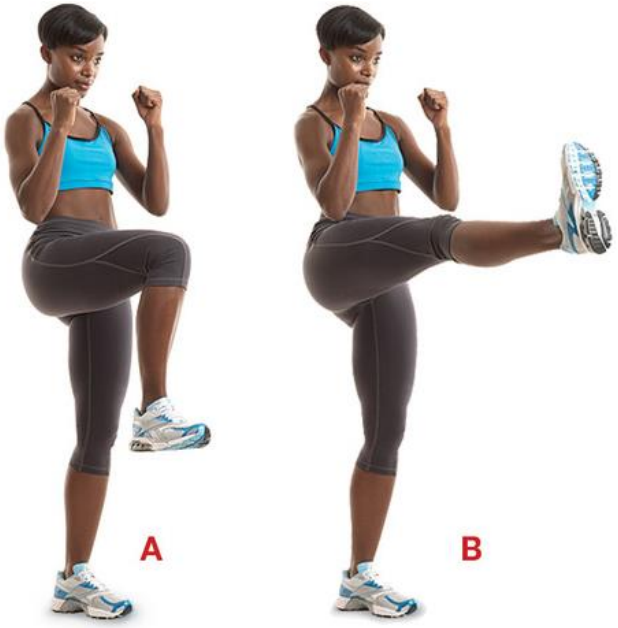
3.1.2 SELECT with SUM aggregation and multiple JOINs and GROUP BY

What is the course title and sum of course credits for each course taught by instructor Brandt?

3.1.3 DELETE with NOT IN and nested SELECT

Delete all courses that have never been offered (that is, do not occur in the Section table).

Solutions to Demo Exercises



Advanced SQL Queries and Data Manipulation

The following SQL queries should be run against the University database.

3.1.1 SELECT with COUNT aggregation and GROUP BY

Find the number of enrolments of each course section that was offered in Fall 2009.

```
SELECT CourseID, SectionID, Count(StudID)
FROM Takes
WHERE Semester = 'Fall' AND StudyYear = 2009
GROUP BY CourseID, SectionID;
```

3.1.2 SELECT with SUM aggregation and multiple JOINs and GROUP BY

What is the course title and sum of course credits for each course taught by instructor Brandt?

```
SELECT Title, SUM(Credits)
FROM Instructor
        NATURAL JOIN Teaches
        NATURAL JOIN Course
WHERE InstName = 'Brandt'
GROUP BY Title;
```

Title	SUM(Credits)
Game Design	8
Image Processing	3

3.1.3 DELETE with NOT IN and nested SELECT

Delete all courses that have never been offered (that is, do not occur in the Section table).

```
DELETE FROM Course
WHERE CourseID NOT IN
(Select CourseID FROM Section);
```

PS. Run the UniversityDB.sql script again to restore tables to the original instance.

Exercises

Please answer all exercises to demonstrate your Database Skills.

MySQL Exercises

Solutions are available at 11:45

Advanced SQL Queries and Data Manipulation

Exercises

SQL queries should be run against the University database.

Re-run the UniversityDB Script to restore tables to initial instances.

Have a print of the University Database Schema Diagram and the Database Instance readily available.

3.2.1 SELECT with MAX aggregation

Find the highest salary of any instructor

3.2.2 Scalar Subquery

Find all instructors earning the highest salary. There might be more than one with the same salary.

3.2.3 DELETE using IN

Delete courses BIO-101 and BIO-301 in the Takes table.

After this question run the UniversityDB Script to restore tables to initial instances.

3.2.4 Advanced WHERE condition using NOT IN

Find those students who have not taken a course.

3.2.5 Advanced WHERE condition using ALL

Find the name(s) of those department(s) which have the highest budget, i.e. a budget which is higher than or equal to those for all other departments.

3.2.6 Advanced WHERE condition using SOME

Find the names of those students who have the same name as some instructor. Use the SOME operator for this.

Make another statement querying the same, but without using SOME.

Create Table & Advanced SQL Queries

3.2.7 Create and populate a table GradePoints(Grade, Points) to provide a conversion from letter grades to numeric scores such that

SELECT * FROM GradePoints;

gives:

Grade	Points
A	4.0
A-	3.7
B	3.0
B+	3.3
B-	2.7
C	2.0
C+	2.3
C-	1.7
D	1.0
D+	1.3
D-	0.7
F	0.0

This shows that an "A" corresponds to 4 points, an "A-" to 3.7 points, and so on.

The Grade-Points earned by a student for a course offering (section) is the number of credits for the course multiplied by the points for the grade that the student received.

The *Total* Grade-Points earned by a student is the sum of grade points for all courses taken by the students.

3.2.8 Find the Total Grade-Points earned by each student who has taken a course. Hint: use **GROUP BY.**

3.2.9 Find the Total Grade-Points Average (GPA) for each student who has taken a course, that is, the total grade-points divided by the total credits for the associated courses. Order the students by falling averages.

Hint: use **GROUP BY.**

Create Table & Advanced SQL Queries

3.2.10 Query using UNION

Now modify the queries from the two previous questions such that students who have not taken a course are also included in the result.

Hint: use the UNION operator.

3.2.11 Testscores Example

Create a relation schema Testscores (by a table declaration) and insert values such that

SELECT * FROM Testscores;

gives:

Student	Test	Score
Brandt	A	47
Brandt	B	50
Brandt	C	NULL
Brandt	D	NULL
Chavez	A	52
Chavez	B	45
Chavez	C	53
Chavez	D	NULL

Then find the maximal score for each student who has an average larger than 49.

Then find those students for whom some score is unknown.